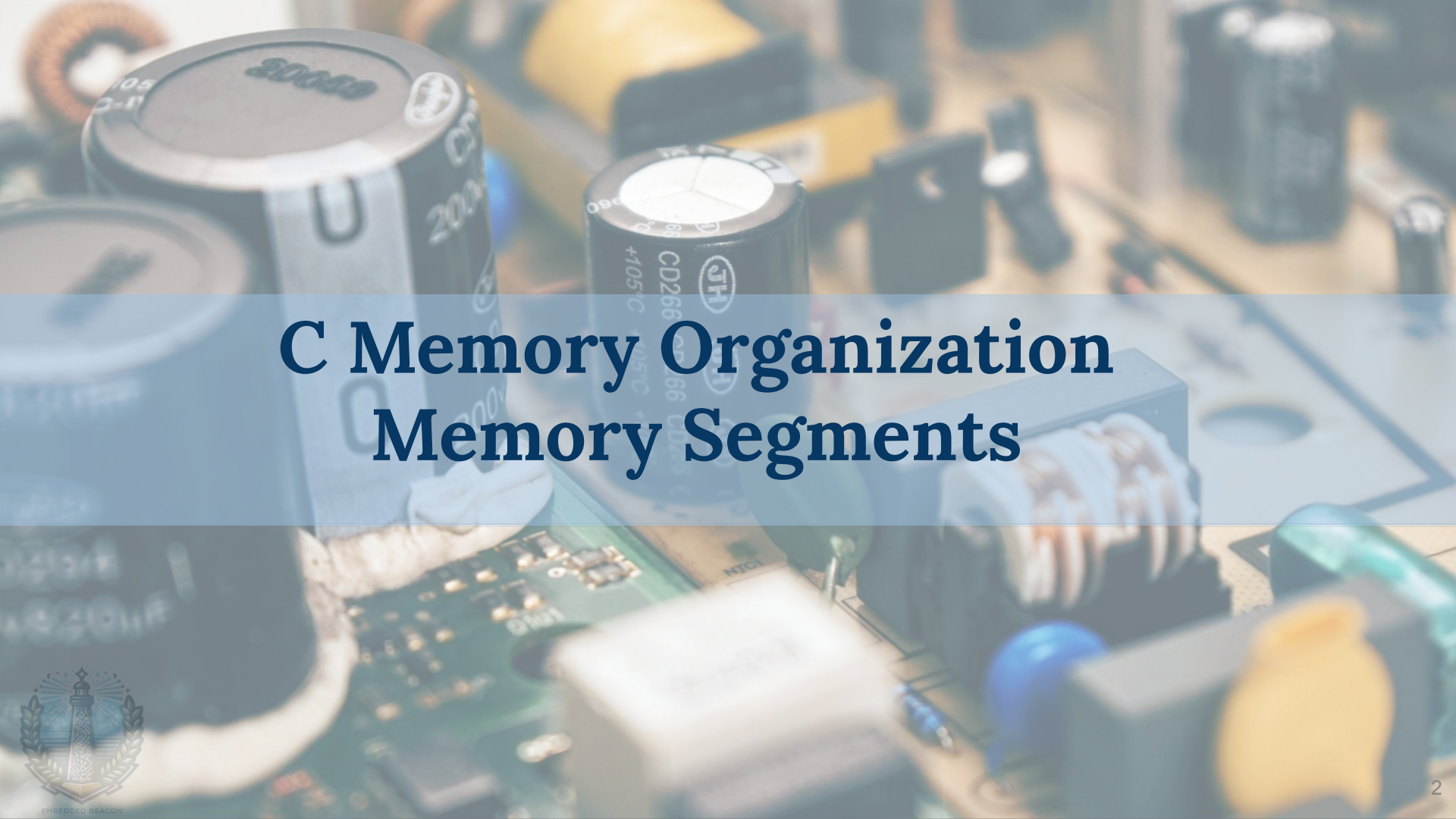
Embedded C Programming







C Memory Organization
Memory Segments

Introduction

- Memory management is critical for C programmers and requires a solid understanding of memory structures.
- This knowledge is essential not only for C but also for languages like C++ and Java.
- In C, memory management is manual, with the programmer responsible for both allocation and deallocation.
- This contrasts with high-level languages like Java or C#, where a garbage collector handles deallocations.
- In C and C++, lacking a garbage collector makes understanding memory management crucial.
- This session and the next session will explore memory-related topics to provide a foundational understanding for these languages.

Contents to be discussed:

Throughout this sub-session:

- We examine a process's typical memory structure to understand memory interaction.
- We discuss static and dynamic memory layouts.
- We introduce the different segments in these layouts, noting which are in the executable file and which are created during process loading.
- We present tools and commands to detect these segments and view their contents in both object files and running processes.
- We learn about the Stack and Heap segments, which are part of the dynamic memory layout where all allocations and deallocations occur.

Let's begin by discussing the process memory layout to understand how a running process's memory is segmented and the purpose of each segment.

Process memory layout

- Whenever you run an executable file, the operating system creates a new process.
- A process is a running program in memory with a unique Process Identifier (PID).
- The operating system is responsible for starting and loading new processes.
- A process runs until it exits normally or receives a signal like SIGTERM, SIGINT, or SIGKILL. SIGTERM and SIGINT can be ignored by the process, but SIGKILL will immediately and forcefully terminate it.

1. SIGTERM: This termination signal allows the process to clean up before exiting.
2. SIGINT: This interrupt signal is usually sent to the foreground process by pressing Ctrl + C.
3. SIGKILL: This kill signal forcefully closes the process without allowing it to clean up.

Memory Layout

- When a process is created, the operating system first allocates a portion of memory for it and then applies a predefined memory layout.
- This layout is generally similar across different operating systems, especially Unix-like ones.
- In this session, we will explore this memory layout and introduce several important and useful terms.



Memory Layout

The memory layout of a typical process is divided into several parts, each called a segment.

Each segment is a region of memory with a specific purpose and is designed to store a particular type of data. The main segments in a running process are:

- Uninitialized data segment (BSS segment)
- Data segment
- Text segment (Code segment)
- Stack segment
- Heap segment

We will examine each segment and discuss their roles in program execution, focusing in detail on the Stack and Heap segments. Before that, we will introduce tools to inspect memory.

Memory Architecture

C Program

Compiled

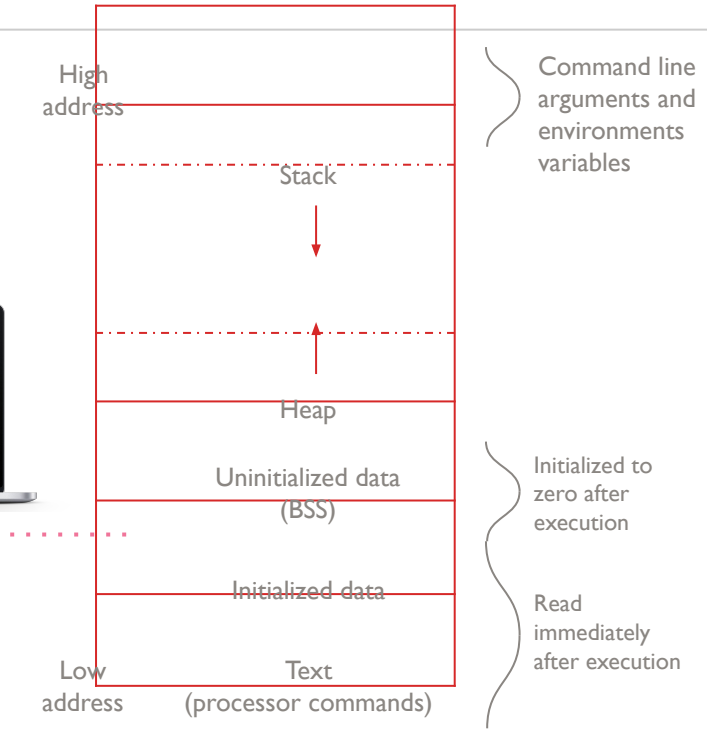
Binary Executable File

Executed

RAM

Computer access instructions from RAM





Memory Layout

Discovering memory structure

- Unix-like operating systems provide tools to inspect a process's memory segments. Some segments come from the executable file, while others are created at runtime.
- An executable file contains machine instructions from the compiler.
- A process is the running program in memory executed by the CPU. The executable file is the basis for creating processes, which are managed by the operating system.
- ***In a running process, segments from the executable file form the static memory layout, while segments created at runtime form the dynamic memory layout.***

Static Vs Dynamic Memory Layouts

- Static and dynamic memory layouts both have defined segments.
- ***The static memory layout is created by the compiler and fixed in the executable file*** when the source code is compiled. This layout can be understood by examining the source code or compiled file.
- ***The dynamic memory layout is created as the process runs,*** with memory allocated and changed according to the program's needs.
- Unlike static memory, dynamic memory content can't be predicted without running the program, and each run can produce different layouts.
- To understand dynamic memory, you need to observe the running process. Let's start by examining the static memory layout.

Probing static memory layout

- The tools used for inspecting the static memory layout usually work on the object files.
- To get some initial insight, we'll start with an example, which is a minimal C program that doesn't have any variable or logic as part of it:

```
int main(int argc, char** argv) {
  return 0;
}
```



Size Command

The **size command** is the first tool we will introduce. It can be used to display the static memory layout of the executable file.

You can use the `size` command to view the different segments that make up the static memory layout, as shown below:

```
$ size ex4_1-linux.out
   text    data     bss     dec     hex   filename
   1099     544       8    1651     673   ex4_1-linux.out
$
```

As you can see, the static layout includes the Text, Data, and BSS segments. The sizes shown are in bytes.

BSS segment

- Let's begin with the BSS segment, which stands for ***Block Started by Symbol.*** This term originally referred to reserved regions for uninitialized data. Today, it is used for uninitialized global variables or global variables set to zero.
- We will expand previous example by adding some uninitialized global variables. You will see that these variables will be included in the BSS segment. Example, shown in the following code box, demonstrates this.

```
int global_var1;
int global_var2;
int global_var3 = 0;

int main(int argc, char** argv) {
  return 0;
}
```

BSS Segment

- When you compare the previous output with that from example 1, you will see that the size of the BSS segment has increased.
- This is because declaring global variables that are uninitialized or set to zero adds to the BSS segment.
- These global variables are part of the static layout, preallocated when the process loads, and they remain allocated for the entire lifetime of the process.
- In other words, they have a static lifetime.

Note:

- For design reasons, it's generally better to use local variables in algorithms.
- Excessive global variables can increase the binary size and pose security risks by exposing sensitive data.
- Additionally, global variables can lead to concurrency issues like data races, namespace pollution, unclear ownership, and other complications.

Note

- When allocating memory, the exact number of bytes a program needs doesn't matter.
- The allocator acquires memory in memory pages until the total meets the program's needs.
- In Shell Box shown, the __common section in the _DATA segment is 12 bytes.
- This corresponds to the BSS segment, which isn't explicitly shown as BSS.
- It represents three uninitialized global integer variables, totaling 12 bytes (each integer is 4 bytes).
- Uninitialized global variables are automatically set to zero by default.

EMBEDDED BEACON

Data segment

Next, let's discuss the Data segment in the static memory layout.

To illustrate the types of variables stored in the Data segment, we'll declare additional global variables, but this time we'll initialize them with nonzero values.

Example 3 builds on example 2 by adding two new initialized global variables:

```
int global_var1;
int global_var2;
int global_var3 = 0;

double global_var4 = 4.5;
char global_var5 = 'A';

int main(int argc, char** argv) {
  return 0;
}
```

Data Segment

```
void func()
{
static int i;
static int j = 1;
...
}
```

- The Data segment stores initialized global variables with non-zero values. Comparing the output of the size command for examples 2 and 3, you can see the Data segment increased by 9 bytes.
- This change corresponds to the sizes of two new global variables: an 8-byte double and a 1-byte char.
- ***On Linux, you can use readelf and objdump to view ELF files and inspect the static memory layout.***
- Functions can also have static variables that retain their values across multiple calls. These variables are stored in the Data segment or BSS segment, depending on the platform and whether they are initialized.
- The example code box shows how to declare static variables within a function:



objdump

At runtime, func accesses these variables stored in the Data or BSS segment, both with static lifetimes. The initialized j is in the Data segment, while the uninitialized i is in the BSS segment.

Next, we will introduce a command to examine the BSS segment.

On Linux, the objdump command displays the contents of memory segments in an object file.

On macOS, the equivalent command is gobjdump, which needs to be installed first.



objdump

The code above is straightforward; it declares three global variables with initial values. After compilation, we need to examine the Data segment to find these written values.

The following commands will show you how to compile the code and use objdump to view the content of the Data segment:

```
$ gcc ExtremeC_examples_chapter4_4.c -o ex4_4.out
```

```
$ objdump -s -j .data ex4_4.out

a.out:     file format elf64-x86-64

Contents of section .data:
 601020 00000000 00000000 00000000 00000000  ................
 601030 21000000 67341512 41424344 4500      !....4..ABCDE.
$
```



Memory Sections

To read the output of the .data section:

- The first column shows the address.
- The next four columns display the data, with each showing 4 bytes, so each row represents 16 bytes.
- The last column provides the ASCII representation, with dots (.) for non-alphanumeric characters.

The -s option tells objdump to display the full content, and -j .data specifies to show the .data section.

- The first line shows 16 bytes of zeros, indicating no variables are stored there.
- The second line starts at address 0x601030 and shows the Data segment contents: the first 4 bytes are for the x variable, the next 4 bytes for y, and the remaining 6 bytes for the z array, with its contents visible in the last column.

Endianness

- Values are stored differently than written due to byte-swapping. For instance, the decimal value 33 (0x00000021) is stored as 0x21000000, and 0x12153467 for y is stored as 0x67341512.
- The differences in value storage are due to endianness: big-endian and little-endian.
- In big-endian format, the largest byte (0x12) comes first, so 0x12153467 is stored as 0x12 0x15 0x34 0x67. In little-endian format, the smallest byte (0x67) comes first, so 0x12153467 is stored as 0x67 0x34 0x15 0x12.
- ***C handles values correctly regardless of endianness.***
- ***Endianness is determined by the CPU***, so different CPUs may use different byte orders, which can affect whether executables run on different hardware.



Text segment

- The Text segment, or Code segment, holds these instructions and is part of the static memory layout.
- The processor fetches and executes these instructions at runtime.
- To explore this, let's examine the Text segment of a real executable file. We'll use a new example, shown in code box 5, which features a simple, empty main function:



Probing dynamic memory layout

The dynamic memory layout is the runtime memory of a process, existing only while the process runs. When you execute an executable file, the loader creates a new process and sets up the initial dynamic memory layout. This includes segments from the static layout of the executable file and adds two new segments: ***the Stack and Heap.***

In total, a process's memory layout includes five segments:

- Three segments are from the static layout in the executable.
- The Stack and Heap segments are dynamic and exist only during runtime.

Memory mappings

- Let's start with a simple example that runs indefinitely, allowing us to inspect its memory. Once we're done, we can terminate the process. Here's the example code:
- The code runs in an infinite loop to keep the process active for memory inspection.
- To build and run it on Linux:

```
#include <unistd.h> // Needed for sleep function

int main(int argc, char** argv) {
  // Infinite loop
  while (1) {
    sleep(1); // Sleep 1 second
  };
  return 0;
}
```

Linux commands

To build and run it on Linux:

```
$ gcc ExtremeC_examples_chapter4_6.c -o ex4_6.out
$ ./ex4_6.out &
```

The process runs in the background, and the PID (e.g., 402) is used for termination. The PID changes with each run.

To stop the process, use:

```
$ kill -9 402
```

On Linux, process information is available in `/proc` through a special filesystem called procfs. This allows us to view memory mappings, including the Stack and Heap segments.

To view the memory mappings of a process, list the contents of the `/proc/[PID]` directory:

```
$ ls /proc/402
```



Stack segment

The Stack is a vital part of dynamic memory, represented as [stack] in memory mappings. Both the Stack and Heap have dynamic content that changes during runtime, often requiring real-time inspection with a debugger like gdb.

The Stack segment is limited in size and not suitable for large objects. If it overflows, usually due to too many function calls or local variables, the operating system terminates the process.

Here's how the Stack works:

- Local variables are stored on the Stack. When a function is called, a stack frame is added, and local variables are pushed onto the Stack.
- When the function exits, the stack frame is removed, and variables are popped off the Stack.



Stack Segments

The Stack operates as a Last In, First Out (LIFO) structure. Proper management is crucial to avoid crashes due to stack overflow.

The Stack segment also manages function calls and local variables. Due to its private and sensitive nature, examining the Stack requires a debugger.

```
void func() {
  // The memory required for the following variable is
  // allocated from the stack segment.
  int a;
  ...
}
```

Next, we'll focus on the Heap segment and return to the Stack in the debugging chapter.

Heap Segment

```c
#include <unistd.h> // Needed for sleep function
#include <stdlib.h> // Needed for malloc function
#include <stdio.h> // Needed for printf

int main(int argc, char** argv) {
  void* ptr = malloc(1024); // Allocate 1KB from heap
  printf("Address: %p\n", ptr);
  fflush(stdout); // To force the print
  // Infinite loop
  while (1) {
    sleep(1); // Sleep 1 second
  };
  return 0;
}
```

```
$ g++ ExtremeC_examples_chapter4_7.c -o ex4_7.out
$ ./ex4_7.out &
[1] 3451
Address: 0x19790010
$
```

Key Points - Heap

- **Heap Allocation**: The malloc function is used to allocate memory from the Heap. It returns a pointer to the allocated memory.
- **Pointer Casting**: The generic pointer returned by malloc should be cast to a specific type before use.
- **Inspecting Heap Memory**: After running the example, you can check the memory mappings of the process to find the Heap segment, which will be marked as [heap].
- **Memory Size**: The allocated memory might be larger than requested to optimize future allocations. For example, requesting 1 KB might allocate 132 KB.
- **Pointer Usage**: The address returned by malloc (e.g., 0x19790010) will be within the Heap's address range.
- **Heap Growth**: The Heap can grow significantly and is used for large or permanent objects.
- **Pointer Management**: Local pointers are allocated on the Stack but point to memory on the Heap. Managing memory correctly to avoid leaks is crucial. Use the free function to deallocate memory and prevent leaks.

Example

Example Code:

- **Memory Allocation**: malloc(1024) allocates 1 KB.
- **Pointer Casting**: Cast the returned pointer to char* for use.
- **Memory Deallocation**: Call free to deallocate memory before exiting.

Next, we'll dive deeper into the Stack and Heap segments in the following session.



Summary

In this session, we covered the following key topics about process memory structures in Unix-like systems:

1. **Memory Structures Overview**: Explained both dynamic (runtime) and static (executable) memory layouts.
2. **Static Memory Layout**:
    - **Text Segment**: Contains machine-level instructions.
    - **BSS Segment**: Holds uninitialized global variables.
    - **Data Segment**: Stores initialized global variables.
    - **Tools**: Used size, objdump, and mentioned readelf to explore these segments.
3. **Dynamic Memory Layout**:
    - Includes all static segments plus two new segments: Stack and Heap.
    - **Stack Segment**: Used for local variable storage and function calls.
    - **Heap Segment**: Used for dynamic memory allocation with functions like malloc and free.
4. **Memory Leakage**: Discussed how improper management of Heap memory can lead to leaks.

In the next chapter, we will delve deeper into the Stack and Heap segments, providing more examples and tools for memory probing and management.



Thank you